

Arquitetura e Organização de Computadores

Projeto do Banco de Registradores do RISC-V (XREGS)

1. Identificação do Aluno

Nome: Victor Enéias Oliveira

Matrícula: 221038364

Turma: 2

2. Objetivo

Projetar, simular e verificar um banco de registradores similar ao utilizado no processador RISC-V, contendo 32 registradores de 32 bits, dois barramentos de leitura (ro1 e ro2), um barramento de escrita (data) e o registrador de índice zero constante em zero. A verificação foi realizada por meio de um testbench em VHDL utilizando o comando ASSERT.

3. Descrição do Projeto

O banco de registradores foi modelado como uma pequena memória. Para isso, foi criado um tipo de dados com os comandos `type` e `array`, representando um vetor de 32 palavras de WSIZE bits, instanciado como o sinal `breg`:

```
type reg_array is array (0 to 31) of std_logic_vector(WSIZE-1 downto 0);
signal breg : reg_array := (others => (others => '0'));
```

A escrita é síncrona: ocorre na borda de subida do relógio (`rising_edge(clk)`) somente quando o sinal de habilitação `wren` está em '1'. As leituras são assíncronas (combinacionais), de forma que as saídas `ro1` e `ro2` refletem imediatamente o conteúdo dos registradores endereçados por `rs1` e `rs2` — comportamento necessário para que o processador leia os operandos no mesmo ciclo de clock.

4. Código VHDL do XREGS

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity XREGS is
    generic (WSIZE : natural := 32);
    port (
        clk, wren : in std_logic;
        rs1, rs2, rd : in std_logic_vector(4 downto 0);
        data : in std_logic_vector(WSIZE-1 downto 0);
        ro1, ro2 : out std_logic_vector(WSIZE-1 downto 0)
    );
end XREGS;

architecture behavioral of XREGS is

    -- Tipo de dados: vetor de 32 palavras de WSIZE bits
    type reg_array is array (0 to 31) of std_logic_vector(WSIZE-1 downto 0);

    -- Banco de registradores, inicializado com zeros
    signal breg : reg_array := (others => (others => '0'));

begin

    -- -----
```

```

process (clk)
begin
    if rising_edge(clk) then
        if wren = '1' and rd /= "00000" then
            breg(to_integer(unsigned(rd))) <= data;
        end if;
    end if;
end process;

-- -----

ro1 <= (others => '0') when rs1 = "00000" else
    breg(to_integer(unsigned(rs1)));

ro2 <= (others => '0') when rs2 = "00000" else
    breg(to_integer(unsigned(rs2)));

end behavioral;

```

5. Código VHDL do Testbench

O testbench realiza as três atividades de verificação solicitadas:

- Preenche o banco de registradores com uma sequência de valores (valor $i \times 10$ no registrador i), utilizando um laço for;
- Lê todos os valores escritos pelas duas saídas (ro1 e ro2) e verifica a correção com ASSERT, reportando mensagem de erro com SEVERITY error em caso de falha;
- Escreve um valor diferente de zero (0xDEADBEEF) no registrador zero e verifica, também com ASSERT, que o seu valor não é alterado.

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity tb_XREGS is
end tb_XREGS;

architecture test of tb_XREGS is

    constant WSIZE : natural := 32;
    constant CLK_PERIOD : time := 10 ns;

    signal clk : std_logic := '0';
    signal wren : std_logic := '0';
    signal rs1, rs2, rd : std_logic_vector(4 downto 0) := (others => '0');
    signal data : std_logic_vector(WSIZE-1 downto 0) := (others => '0');
    signal ro1, ro2 : std_logic_vector(WSIZE-1 downto 0);

    signal fim : boolean := false; -- sinaliza fim da simulacao

begin

    -- Instancia do DUT (Device Under Test)
    DUT: entity work.XREGS
        generic map (WSIZE => WSIZE)
        port map (
            clk => clk,
            wren => wren,
            rs1 => rs1,
            rs2 => rs2,
            rd => rd,
            data => data,
            ro1 => ro1,
            ro2 => ro2
        );

    -- Geracao do clock (para quando fim = true)
    clk_gen: process
    begin
        while not fim loop

```

```

        clk <= '0';
        wait for CLK_PERIOD / 2;
        clk <= '1';
        wait for CLK_PERIOD / 2;
    end loop;
    wait;
end process;

-- Processo de estimulo e verificacao
stim: process
    variable valor_esperado : std_logic_vector(WSIZE-1 downto 0);
begin

    wren <= '1';
    for i in 0 to 31 loop
        rd <= std_logic_vector(to_unsigned(i, 5));
        data <= std_logic_vector(to_unsigned(i * 10, WSIZE));
        wait until rising_edge(clk);
        wait for 1 ns; -- pequena folga apos a borda
    end loop;
    wren <= '0';

    for i in 0 to 31 loop
        rs1 <= std_logic_vector(to_unsigned(i, 5));
        rs2 <= std_logic_vector(to_unsigned(i, 5));
        wait for 2 ns; -- tempo para a leitura combinacional

        if i = 0 then
            -- registrador 0 deve ser sempre zero
            valor_esperado := (others => '0');
        else
            valor_esperado := std_logic_vector(to_unsigned(i * 10, WSIZE));
        end if;

        assert ro1 = valor_esperado
            report "ERRO em ro1: registrador " & integer'image(i) &
                " leu valor incorreto"
            severity error;

        assert ro2 = valor_esperado
            report "ERRO em ro2: registrador " & integer'image(i) &
                " leu valor incorreto"
            severity error;
    end loop;

    wren <= '1';
    rd <= "00000";
    data <= x"DEADBEEF";
    wait until rising_edge(clk);
    wait for 1 ns;
    wren <= '0';

    rs1 <= "00000";
    rs2 <= "00000";
    wait for 2 ns;

    assert ro1 = std_logic_vector(to_unsigned(0, WSIZE))
        report "ERRO: registrador 0 foi alterado (ro1)!"
        severity error;

    assert ro2 = std_logic_vector(to_unsigned(0, WSIZE))
        report "ERRO: registrador 0 foi alterado (ro2)!"
        severity error;

    report "Fim da simulacao: todos os testes executados."
        severity note;

    fim <= true;
    wait;
end process;

end test;

```

6. Resultados da Simulação

O projeto foi compilado e simulado, e nenhuma das verificações com ASSERT reportou erro. Ao final da execução, foi emitida a mensagem de conclusão:

```
tb_XREGS.vhd:125:9:@388ns:(report note): Fim da simulacao: todos os testes executados.
```

7. Qual a opção usada para simular a constante zero em XREGS[0]?

Foram combinadas duas técnicas complementares:

1. Bloqueio da escrita: no processo de escrita síncrona, a condição `rd /= "00000"` faz com que qualquer escrita endereçada ao registrador 0 seja ignorada. Como o banco é inicializado com zeros na declaração do sinal, o registrador 0 nunca deixa de valer zero. Esta é a técnica essencial para garantir a constante.

2. Leitura forçada em zero: nas atribuições combinacionais das saídas, a condição `when rs1 = "00000"` (e o equivalente para `rs2`) retorna diretamente o valor zero, sem consultar o banco. Trata-se de uma proteção adicional que, além de robustez, permite ao sintetizador eliminar os flip-flops do registrador 0, economizando recursos de hardware.

Dessa forma, escritas no registrador 0 não têm efeito e leituras desse registrador sempre retornam zero, conforme especificado na arquitetura RISC-V.

8. Conclusão

O banco de registradores XREGS foi projetado conforme a especificação, com escrita síncrona habilitada por `wren`, duas portas de leitura assíncronas e o registrador zero constante. O testbench, utilizando ASSERT, validou a escrita e leitura de todos os 32 registradores por ambas as saídas e confirmou a imutabilidade do registrador 0, sem nenhum erro reportado na simulação.